

PROJECT DOCUMENTATION

CODING ASSIGNMENT – SIPLACE SETUP REPORT GENERATOR

OBJECTIVE

**C# WINDOWS FORMS APPLICATION FOR SIPLACE
XML RECIPE IMPORT, SQL SERVER STORAGE AND
EXCEL/PDF SETUP REPORT GENERATION.**

DATE: 24-09-2026

Task assigned by:

Mr. Ilavarasan

Report submitted by:

Chethan Nazre S

1. PROJECT OVERVIEW:

The project is a C# Windows Forms application developed for working with SIPLACE XML recipe files. The application first reads the recipe data then saves the extracted information in SQL Server and allows users to generate setup reports in Excel and PDF.

Implemented workflow

XML recipe selection → XML parsing → SQL Server storage → imported recipe listing → recipe selection → setup-data grouping → Excel/PDF report generation.

2. OBJECTIVE:

The objective of the assignment is to create a Windows Application that can Import SIPLACE XML file, store the recipe and setup details on SQL Server and Generate Report.

The implemented workflow is shown below: Importing an XML recipe, Data Extraction, Database Storage, Recipe Selection, Grouping of setup Data and Report Generation in both Excel and PDF.

3. TECHNOLOGIES USED:

Technology Requirements

C#, .NET 6/7/8, WinForms or WPF, SQL Server Express, Entity Framework Core, LINQ to XML, ClosedXML, QuestPDF.

Technology	Purpose
C# /.NET	Application Development
Windows Forms	Desktop User Interface
SQL Server Express	Persistent Data Storage
Entity Framework Core	Database Access and Object-Relational Mapping
LINQ to XML	SIPLACE XML Parsing
ClosedXML	Excel Report Generation
QuestPDF	PDF Report Generation
Visual Studio	Development And Build Environment

4. APPLICATION STRUCTURE:

The project files are divided based on their purpose. The Models folder contains the Recipe and Setup Classes. Database access is handled through SiplaceContext, while the XML parsing and report generation are placed under Services. MainForm.cs contains the WinForms interface Logic

File / Folder	Responsibilities
Models/Recipe.cs	Recipe entity and SetupDetail relationship
Models/Setupdetail.cs	Individual setup/placement entity
Data/SiplaceContext.cs	EF Core SQL Server Context and Relationship Configuration
Services/XMLParser.cs	Loads and parses SIPLACE XML
Services/ReportGenerator .cs	Groups Setup Details and Calculates Quantities
Services/ExcelExporter.cs	Creates Excel Setup Reports
Services/PdfExporter.cs	Creates PDF setup reports
SQL/CreateTables.sql	Database and Table Creation Script
Samples/	Sample and Test Xml Files
Reports/	Generated Excel/PDF Reports
MainForm.cs	WinForms Application Logic

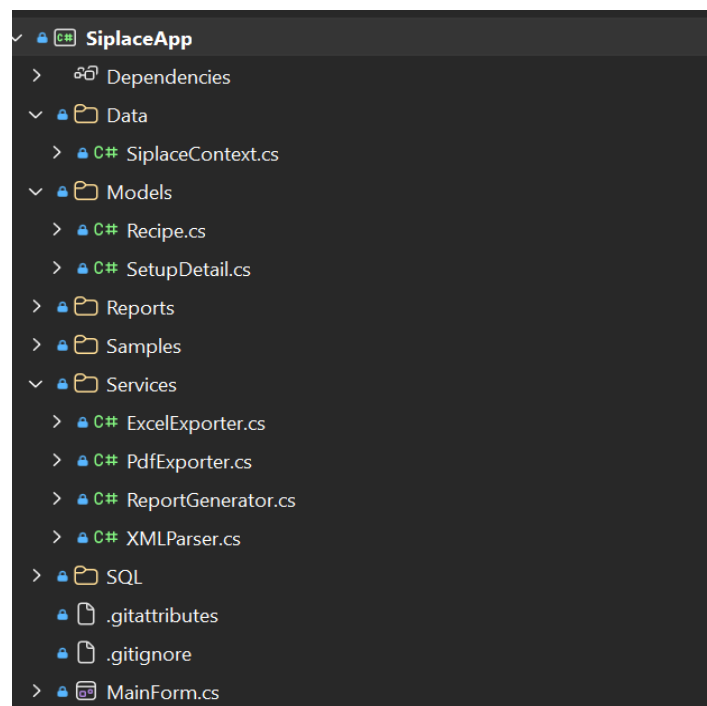


Figure 1: Folder Structure of the Siplace Application

5. SYSTEM WORKFLOW:

1. Select one or more XML files.
2. Parse recipe and setup information with LINQ to XML.
3. Store the recipe and setup records in SQL Server using Entity Framework Core.
4. Display imported recipes in the WinForms DataGridView.
5. Select a recipe.
6. Group setup details by Machine Name, Table, Track, Part Number, and Feeder Type.
7. Generate an Excel or PDF report.

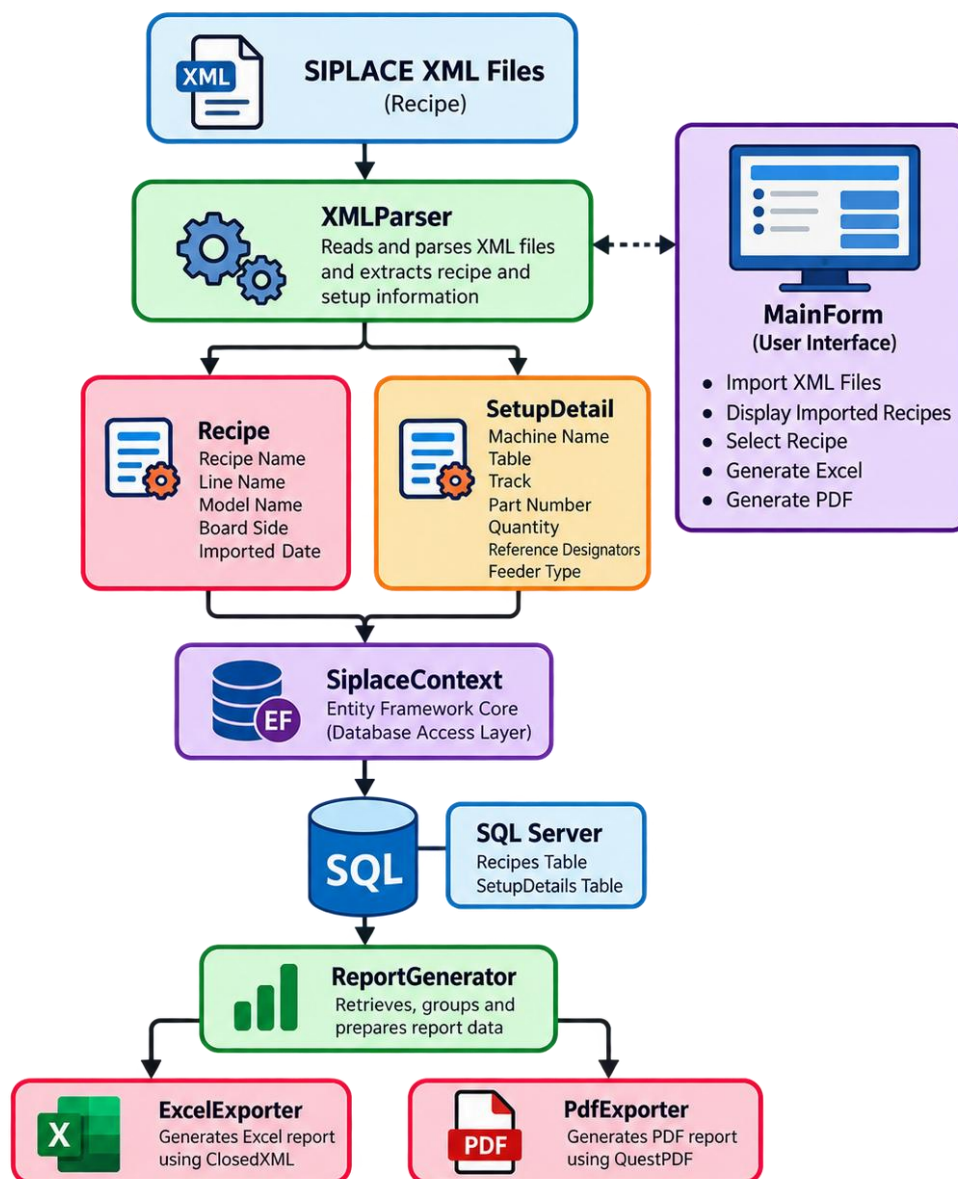


Figure 2: SIPLACE Setup Report Generator System Workflow

6. XML PARSING:

1. Recipe name is read from the XML root Name attribute.
2. Line name is read from the Line element.
3. Model is read from the board name attribute.
4. The current implementation sets Board Side to right for the supplied Sample structure.

For each Pick-up/pair position, the application stores Machine Name, Table, Track, Part Number, Reference Designator, Type and Quantity. The initial quantity of each Setup record is set to 1.

```
XDocument doc = XDocument.Load(filePath);
XElement root = doc.Root;

var recipe = new Recipe
{
    RecipeName = root.Attribute("Name")?.Value ?? "Unknown",
    ImportedDate = DateTime.Now,
    SetupDetails = new List<SetupDetail>()
};
```

Figure 3: The application loads SPLICE XML file using XDocument.

```
var board = root.Descendants("Board").FirstOrDefault();
if (board != null)
{
    recipe.ModelName = board.Attribute("Name")?.Value ?? "";
}
recipe.BoardSide = "Right";
```

Figure 4: The Board element is searched in the XML and its Name attribute is stored as the recipe's ModelName.

7. DATABASE DESIGN:

SQL Server is used in this application to store the imported recipe information and setup details.

Table	Important Columns	Purpose
Recipes	RecipeId, RecipeName, LineName, ModelName, BoardSide, ImportedDate	Stores recipe header information
SetupDetails	DetailId, RecipeId, MachineName, Table, Track, PartNumber, Quantity, ReferenceDesignators, FeederType	Stores setup detail information

```
4 references
public DbSet<Recipe> Recipes { get; set; }
1 reference
public DbSet<SetupDetail> SetupDetails { get; set; }
```

Figure 5: These DbSet properties represent the Recipes and SetupDetails tables in SQL Server.

```
modelBuilder.Entity<SetupDetail>()
    .HasOne(sd => sd.Recipe)
    .WithMany(r => r.SetupDetails)
    .HasForeignKey(sd => sd.RecipeId)
    .OnDelete(DeleteBehavior.Cascade);
```

Figure 6: Recipe and SetupDetail relationship using RecipeId as the foreign key.

8. SETUP DATA GROUPING:

Before report generation, the stored setup records are grouped using the required fields.

The grouped data is used as the input for both the Excel and PDF reports.

Input records	Grouped report result
C101 → PN-100001, quantity 1; C102 → PN-100001, quantity 1	PN-100001 → Quantity 2 → C101, C102
R201 → PN-200001, quantity 1	PN-200001 → Quantity 1 → R201

```
var groupedDetails = _context.SetupDetails.Where
    (d => d.RecipeId == recipeId)
    .GroupBy(d =>
        new { d.MachineName,
              d.Table,
              d.Track,
              d.PartNumber,
              d.FeederType
        })
```

Figure 7: Setup records are grouped by Machine Name, Table, Track, Part Number and Feeder Type.

```
Quantity = g.Sum(d => d.Quantity),
ReferenceDesignators = string.Join(",", g.Select
(d => d.ReferenceDesignators)),
```

Figure 8: The quantity of each group is calculated by adding individual quantities, while reference designators are combined into a single value.

9. REPORT GENERATION:

For the selected recipe, the setup records are retrieved from SQL Server and grouped using Machine Name, Table, Track, Part Number and Feeder Type.

The generated reports contain the recipe name, line name, model, board side, machine name, table, track, part number, quantity, mounting reference designators and feeder type.

9.1 EXCEL REPORT GENERATION:

ClosedXML is used to create the Excel report. A workbook is created with a worksheet named "Setup Report" which writes the recipe information into that worksheet.

```
using var workbook = new XLWorkbook();

var worksheet = workbook.Worksheets.Add("Setup Report");

worksheet.Cell(1,1).Value = "Recipe Name";
worksheet.Cell(1, 2).Value = recipe.RecipeName;

worksheet.Cell(2, 1).Value = "Line Name";
worksheet.Cell(2, 2).Value = recipe.LineName;

worksheet.Cell(3, 1).Value = "Model";
worksheet.Cell(3, 2).Value = recipe.ModelName;

worksheet.Cell(4, 1).Value = "Board Side";
worksheet.Cell(4, 2).Value = recipe.BoardSide;

worksheet.Cell(5, 1).Value = "Imported Date";
worksheet.Cell(5, 2).Value = recipe.ImportedDate.ToString("yyyy-MM-dd HH:mm:ss");
```

Figure 9: Excel Report Generation using ClosedXML

Recipe Name	Sample_Product_A					
Line Name	Sample_Line_01					
Model	Sample_Model_A					
Board Side	Right					
Imported Date	2026-09-23 12:01:50					
Machine Name	Table	Track	Part Number	Quantity	Mounting Reference Designators	Feeder Type
Sample_X4i_S1	Table6	10	PN-200001	1	R201	8mm X
Sample_X4i_S1	Table6	7	PN-100001	2	C101,C102	8mm X

Figure 10: Generated Excel Setup Report

9.2 PDF REPORT GENERATION:

QuestPDF is used for generating the PDF version of the setup report. Then the report is configured in A4 landscape format so that all setup columns can fit on a page.

```
container.Page(page =>
{
    page.Size(PageSizes.A4.Landscape());
    page.Margin(30);
    page.PageColor(Colors.White);
    page.DefaultTextStyle(x => x.FontSize(12));
    page.Header()
        .Text("SIPLACE Setup Report")
        .SemiBold().FontSize(20).FontColor(Colors.Blue.Medium);
    page.Content()
        .PaddingVertical(1, Unit.Centimetre)
        .Column(column =>
        {
            column.Spacing(10);

            column.Item()
                .Text($"Recipe Name: {recipe.RecipeName}").FontSize(14).Bold();

            column.Item()
                .Text($"Line Name: {recipe.LineName}").FontSize(14).Bold();

            column.Item()
                .Text($"Model: {recipe.ModelName}").FontSize(14).Bold();

            column.Item()
                .Text($"Board Side: {recipe.BoardSide}").FontSize(14).Bold();

            column.Item()
                .Text(
                    $"Import Date: {recipe.ImportedDate}").FontSize(14).Bold();
        }
    );
}
```

Figure 11: QuestPDF Implementation for PDF Report Generation

SIPLACE Setup Report						
Recipe Name: Sample_Product_A						
Line Name: Sample_Line_01						
Model: Sample_Model_A						
Board Side: Right						
Import Date: 23-09-2026 12:01:50						
Machine Name	Table	Track	Part Number	Quantity	Reference Designators	Feeder Type
Sample_X4i_S1	Table6	10	PN-200001	1	R201	8mm X
Sample_X4i_S1	Table6	7	PN-100001	2	C101,C102	8mm X

Figure 12: Generated PDF Setup Report

10. USER INTERFACE:

The main window contains the options required for the workflow. XML files can be imported from the Import section, imported recipes are displayed in the list, and Excel or PDF reports can be generated for the selected recipe.

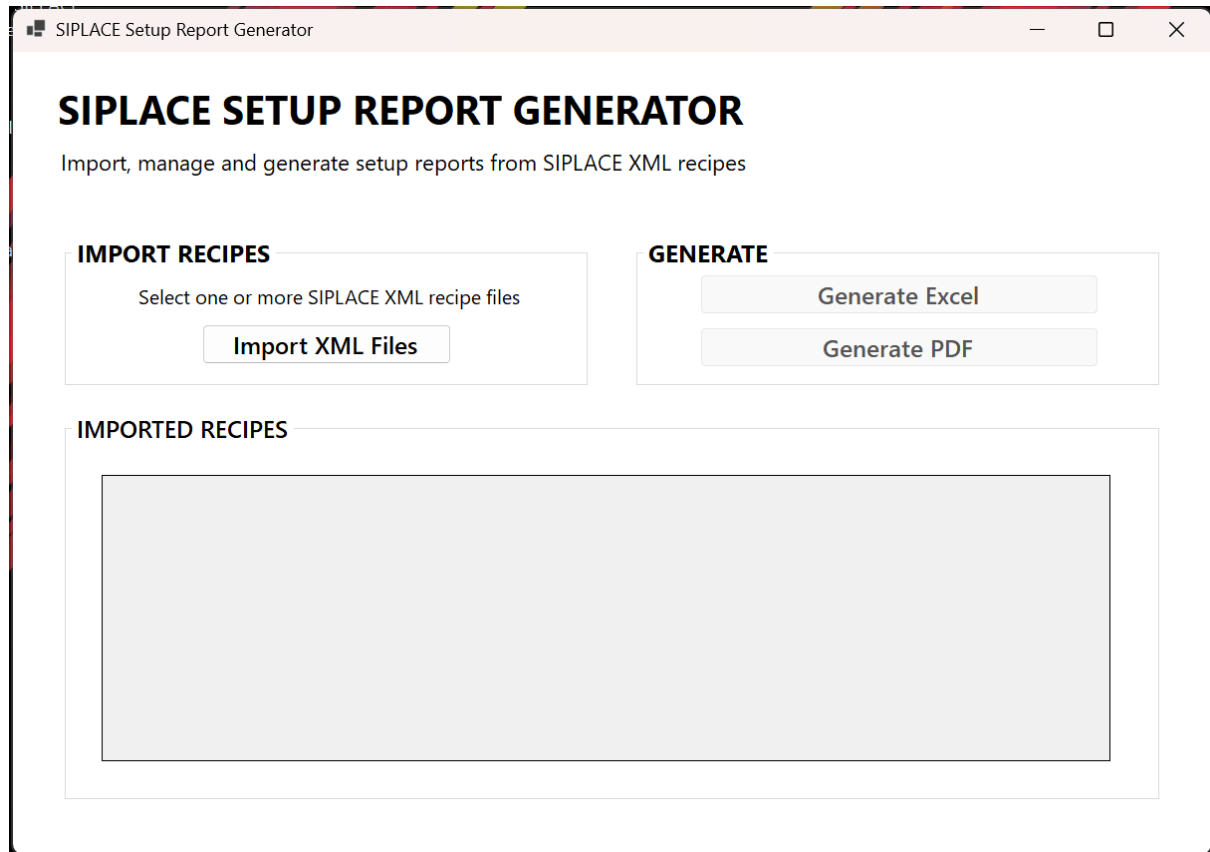


Figure 13: SIPLACE Setup Report Generator User Interface

11. APPLICATION USAGE:

Step 1: Import XML Recipes

Select one or more SIPLACE XML files using the Import option. The selected files are parsed and the extracted recipe and setup details are stored in SQL Server.

Step 2: View Imported Recipes

After importing, the recipes appear in the Imported Recipes list. Select the required recipe from the list before generating the report.

Step 3: Generate Excel Report

Select a recipe and click Generate Excel. The corresponding setup records are retrieved, grouped and written to the Excel report.

Step 4: Generate PDF Report

The Generate PDF button creates the PDF version of the same setup report.

12. TESTING:

The complete application workflow was tested from XML import through report generation. The following test cases were verified:

Test Case	Result
Import sample XML	PASSED
Extract Model from Board	PASSED
Store recipe in SQL Server	PASSED
Store setup details	PASSED
Group duplicate part numbers	PASSED
Generate Excel report	PASSED
Generate PDF report	PASSED
Import multiple XML files	PASSED
Generate report for selected recipe	PASSED
Publish self-contained Windows x64 executable	PASSED
Verification Against Expected Output	PASSED

13. DEPLOYMENT:

The application was prepared for deployment as a self-contained Windows x64 application using the .NET CLI. The Release configuration was used to generate the final deployment package.

- This application was then published using:
dotnet publish -c Release -r win-x64 --self-contained true
- The published application was generated at:
bin\Release\net8.0-windows\win-x64\publish

The publish directory contains application executable and the supporting files required to run the application.

The published build was tested after deployment. The sample XML was imported successfully, and both the Excel and PDF reports were generated from the published application.